TP3 : Github Actions

Exercice 1. Création du repo et premier workflow

1. Créer un dépôt GitHub public nommé tp3-github-actions.
2. Initialiser le dépôt avec un fichier README.md.
3. Cloner le dépôt sur votre machine.
4. Créer le répertoire .github/workflows/.
5. Dans ce répertoire, créer un fichier hello.yml.
6. Ajouter la configuration suivante :

```yaml
name: Premier workflow CI


on:
  push:
  workflow_dispatch:


jobs:
  hello:
    runs-on: ubuntu-latest


    steps:
      - name: Afficher un message
        run: echo "Hello from GitHub Actions"
```

7. Ajouter le fichier à Git, créer un commit et envoyer les modifications sur GitHub.
8. Accéder à l'onglet **Actions** du dépôt.
9. Ouvrir l'exécution du workflow et consulter ses résultats.
10. Ajouter une deuxième étape permettant d'afficher (Utiliser les commandes Linux appropriées.) :
    a. La version de Python disponible sur le runner.
    b. Le répertoire courant.
    c. La liste des fichiers présents dans ce répertoire.

Exercice 2. Déclencheurs et informations d'exécution

1. Modifier le workflow précédent pour lui permettre d'être lancé manuellement.
2. Depuis GitHub, utiliser le bouton **Run workflow**.
3. Comparer cette exécution manuelle avec celle déclenchée précédemment par un `git push`
5. Ajouter une étape affichant les informations suivantes :

```
- name: Informations Git

  run: |

    echo "Branche : $GITHUB_REF_NAME"

    echo "Commit : $GITHUB_SHA"

    echo "Événement : $GITHUB_EVENT_NAME"
```

4. Effectuer un nouveau commit et envoyer les modifications.
5. Consulter les journaux de la nouvelle exécution.
6. Introduire volontairement une étape qui exécute exit 1, puis :
    a. Observer le statut du job.
    b. Identifier l'étape en échec.
    c. Examiner les étapes éventuellement ignorées.
7. Supprimer l'erreur et vérifier que le workflow réussit à nouveau.

Exercice 3. Automatisation des tests et de la couverture de code

1. Reprendre le projet Python réalisé lors du TP2 (Tests logiciels), contenant les différentes fonctions et les tests automatisés avec Pytest.
2. Ajouter les fichiers du projet dans le dépôt GitHub tp3-github-actions, en conservant l'organisation des fichiers et du répertoire tests/.
3. Créer un fichier requirements-dev.txt contenant les dépendances nécessaires au projet:

pytest
pytest-cov

4. Créer un nouveau workflow .github/workflows/ci.yml nommé CI - Tests Python, déclenché automatiquement à chaque push.
5. Ajouter un job nommé tests utilisant un runner ubuntu-latest.
6. Ajouter les étapes suivantes au job :
    a. Récupérer le code du dépôt avec actions/checkout.
    b. Configurer l'environnement Python avec l'action actions/setup-python@v7 :

```yaml
- name: Configurer Python
  uses: actions/setup-python@v7
  with:
    python-version: '3.12'
```

    c. Installer les dépendances du projet à partir du fichier requirements-dev.txt.
    d. Exécuter automatiquement l'ensemble des tests avec Pytest.
    e. Mesurer et afficher la couverture de code à l'aide de pytest-cov.
    f. Configurer la commande de test pour afficher les lignes non couvertes, en utilisant l'option --cov-report=term-missing.
    g. Exemple de commande à adapter au module Python du TP2 :
        - python -m pytest -v --cov=nom_module --cov-report=term-missing
7. Créer un commit et envoyer les modifications sur GitHub.
8. Accéder à l'onglet Actions et consulter les résultats du pipeline :
9. Vérifier que Python et les dépendances ont été correctement installés.
10. Observer le nombre de tests exécutés, réussis et échoués.
11. Consulter le rapport de couverture de code.
12. Identifier les fichiers et les lignes non couverts par les tests.
13. Ajouter un seuil minimal de couverture de 80 % à l'aide de l'option :

        - --cov-fail-under=80
14. Expérimenter avec différents seuils de couverture :
    a. Exécuter le pipeline avec un seuil de 50 %.
    b. Augmenter progressivement le seuil à 70 %, 80 %, 90 %, puis 100 %.
    c. Observer à partir de quel seuil le pipeline échoue.
15. Relancer le pipeline et vérifier les nouveaux résultats.